

Introducción a SQL y análisis de datos — Taller PIP

Esta es una **pre-lectura** para el eje de Análisis de datos del taller. La idea es darte las herramientas básicas para que puedas enfocar el tiempo del taller en resolver el problema, no en entender qué es un `SELECT`.

No hace falta memorizar nada de esto. Es más útil leerlo una vez y volver a mirarlo cuando te trabes con algo puntual, que intentar aprendértelo de memoria antes de escribir una sola consulta. Si más adelante necesitás profundizar, en la carpeta del proyecto hay una **TEORIA.md** más completa.

¿Qué es una base de datos? ¿Y SQL?

Casi toda aplicación guarda su información en una **base de datos**: pedidos de una tienda, vuelos de una aerolínea, alumnos de una facultad. Una base de datos relacional organiza esa información en **tablas**, parecidas a una planilla de cálculo:

- Cada **fila** es un registro (un socio, un vuelo, una venta).
- Cada **columna** es un dato de ese registro (el nombre, la fecha, el monto) y tiene un **tipo** (texto, número...).
- Cada tabla tiene una **clave primaria** (`PRIMARY KEY`, o `PK`): una columna cuyo valor identifica a cada fila de forma única (normalmente un `id`).
- Para relacionar tablas, una columna guarda el `id` de una fila de otra tabla: es una **clave foránea** (`FOREIGN KEY`, o `FK`).

SQL (*Structured Query Language*) es el lenguaje con el que le **hacés preguntas** a una base de datos: "¿cuántos socios hay en Palermo?", "¿quién debe más cuotas?". Vamos a usar **SQLite**, un motor liviano que en el taller corre directamente en tu navegador, sin instalar nada.

Como ejemplo a lo largo de este documento vamos a usar un **club de barrio** inventado:

```
socios —< inscripciones >— actividades
|
└─< cuotas

socios      (id, nombre, barrio, fecha_alta, recomendado_por)
actividades (id, nombre, cupo, precio_mensual)
inscripciones (socio_id, actividad_id)
cuotas      (id, socio_id, mes, monto, fecha_pago)
```

En el club hay 8 socios, 4 actividades (Natación, Fútbol 5, Yoga y Ajedrez) y 16 cuotas. La flecha  se lee "uno a muchos": un socio tiene muchas cuotas.

Cómo vas a trabajar: el SQL Lab

En el taller vas a usar una herramienta hecha para esto, el **SQL Lab**: una sola página web con la base de datos adentro. Se abre con un link (o con doble click en el archivo) y no hay nada que instalar.

Qué ves	Para qué sirve
Panel de esquema (a la izquierda)	Las tablas, sus columnas y tipos, y cuáles son PK y FK . Si hacés click en el nombre de una tabla, se arma un <code>SELECT * ... LIMIT 10;</code> en el editor para espiar cómo son los datos.
Editor	Escribís la consulta. Ctrl/Cmd + Enter la ejecuta. Podés escribir varias sentencias separadas por <code>;</code> .
Resultado	Una tabla con las filas, la cantidad de filas y el tiempo. Si hay un error, te lo explica en español junto con el mensaje original.
Copiar esquema para la IA	Copia un texto listo para pegar en un prompt: el motor (SQLite), las tablas y 3 filas de ejemplo de cada una.
Copiar JSON / CSV / Markdown	Copia el resultado. El JSON sirve para alimentar el dashboard.
Reiniciar base	Vuelve todo al estado original. Podés probar cualquier cosa sin miedo.
Historial	Guarda las consultas que ejecutaste, para recuperarlas con un click.

Tu primera consulta: SELECT, FROM, WHERE, ORDER BY, LIMIT

Una consulta **pide datos**, no modifica nada. Se arma con piezas que siempre se escriben en este orden:

Pieza	Para qué sirve
<code>SELECT columnas</code>	Qué columnas querés ver (<code>*</code> = todas).
<code>FROM tabla</code>	De qué tabla.
<code>WHERE condición</code>	Te quedás solo con las filas que cumplen.
<code>ORDER BY col [DESC]</code>	Cómo ordenás (de menor a mayor por defecto; <code>DESC</code> al revés).
<code>LIMIT n</code>	Cuántas filas como máximo.

```
SELECT nombre, barrio, fecha_alta
FROM socios
WHERE barrio = 'Palermo'
ORDER BY fecha_alta DESC
LIMIT 2;
```

nombre	barrio	fecha_alta
Facundo Luna	Palermo	2024-09-30
Carla Gómez	Palermo	2023-01-20

Se lee casi como una oración: "de los socios, quedate con los de Palermo, ordenalos del más nuevo al más viejo y mostrame los 2 primeros". Los **textos van entre comillas simples**. Con `AS` le podés poner un nombre más lindo a una columna (`COUNT(*) AS socios`).

Resumir: agregaciones, GROUP BY y HAVING

Las **funciones de agregación** convierten muchas filas en un número: `COUNT` (cuántas), `SUM` (suma), `AVG` (promedio), `MIN` y `MAX` .

```
SELECT COUNT(*) AS socios, MIN(fecha_alta) AS el_mas_antiguo, MAX(fecha_alta) AS
el_mas_nuevo
FROM socios;
```

socios	el_mas_antiguo	el_mas_nuevo
8	2021-03-15	2025-03-01

Con `GROUP BY` armás un resumen **por grupo**: una fila por cada valor distinto de la columna.

```
SELECT barrio, COUNT(*) AS socios
FROM socios
GROUP BY barrio
HAVING COUNT(*) ≥ 3;
```

barrio	socios
Caballito	3
Palermo	3

WHERE vs HAVING : WHERE filtra *filas*, antes de agrupar. HAVING filtra *grupos*, después de agrupar (por eso es el único que puede usar COUNT , SUM , AVG ...). Si la condición habla de una cantidad o un promedio, va en HAVING .

Unir tablas: JOIN

La información está repartida: la tabla `inscripciones` solo guarda dos números (`socio_id` y `actividad_id`). Para saber el *nombre* de la actividad hay que ir a buscarlo a `actividades` . Eso hace **JOIN** : **une las filas de dos tablas** cuando se cumple una condición (`ON`), casi siempre "la clave foránea de una es igual a la clave primaria de la otra".

```
SELECT a.nombre AS actividad, COUNT(*) AS inscriptos
FROM inscripciones i
JOIN actividades a ON a.id = i.actividad_id
GROUP BY a.id
ORDER BY inscriptos DESC;
```

actividad	inscriptos
Natación	5
Yoga	4
Fútbol 5	4

Fijate dos cosas. Primero, los **alias** (`i` , `a`): son apodos cortos para las tablas, y se usan como `alias.columna` . Cuando dos tablas tienen una columna con el mismo nombre (como `id`), el alias es obligatorio para que SQLite sepa de cuál hablás. Segundo: **Ajedrez no aparece**, porque nadie se inscribió. El **JOIN** común (`INNER JOIN`) solo deja las filas que tienen pareja en *las dos* tablas.

Si querés conservar *todas* las filas de la tabla de la izquierda aunque no tengan pareja, usás **LEFT JOIN** (las columnas de la derecha quedan en `NULL`):

```
SELECT a.nombre AS actividad, COUNT(i.socio_id) AS inscriptos
FROM actividades a
LEFT JOIN inscripciones i ON i.actividad_id = a.id
GROUP BY a.id
ORDER BY inscriptos;
```

actividad	inscriptos
Ajedrez	0
Fútbol 5	4
Yoga	4
Natación	5

Dos situaciones más que vas a encontrar: a veces una tabla se refiere **a sí misma** (en el club, `recomendado_por` apunta a otro socio) y a veces una misma tabla juega **dos papeles** en la consulta (por ejemplo una tabla de lugares que sirve de origen y de destino). En ambos casos se une la tabla **dos veces con dos alias distintos**, uno por cada papel.

Cuidado: un JOIN puede duplicar filas. Si cada fila de una tabla tiene *varias* parejas en la otra, se repite una vez por cada pareja. Si después sumás una columna de la tabla original, el total sale inflado. Un buen hábito: **comparar el total antes y después de cada JOIN**.

Cuatro cosas que traen sorpresas

1. NULL: la ausencia de dato

`NULL` no es cero ni vacío: significa **"no hay dato"** (por ejemplo, una cuota que todavía no se pagó no tiene `fecha_pago`). Tres reglas: no se compara con `=` sino con `IS NULL` / `IS NOT NULL`; `5 + NULL` da `NULL`; y funciones como `AVG` y `SUM` **ignoran** los `NULL`.

```
-- Devuelve 0 filas SIEMPRE: nada es "igual" a NULL
SELECT * FROM cuotas WHERE fecha_pago = NULL;

-- Esta sí funciona
SELECT * FROM cuotas WHERE fecha_pago IS NULL;
```

2. COUNT(*) no es lo mismo que COUNT(columna)

Forma	Qué cuenta
<code>COUNT(*)</code>	Todas las filas.
<code>COUNT(columna)</code>	Las filas donde esa columna no es NULL .
<code>COUNT(DISTINCT columna)</code>	Los valores distintos.

La resta `COUNT(*) - COUNT(columna)` te dice cuántos `NULL` hay en esa columna: una forma rapidísima de *explorar* los datos.

3. La división entera

En SQLite, si dividís **dos enteros**, el resultado es un entero: se tira la parte decimal.

```
SELECT a.nombre, COUNT(*) AS inscriptos, a.cupo,
       COUNT(*) / a.cupo AS mal,
       1.0 * COUNT(*) / a.cupo AS bien
FROM inscripciones i JOIN actividades a ON a.id = i.actividad_id
GROUP BY a.id;
```

nombre	inscriptos	cupo	mal	bien
Natación	5	20	0	0.25
Fútbol 5	4	10	0	0.4
Yoga	4	4	1	1

La solución: que *uno* de los números sea decimal (multiplicar por `1.0`, o por `100.0` si querés porcentaje). Un porcentaje que da 0 o 1 justo es una señal de alarma.

4. Las fechas son texto

SQLite no tiene un tipo "fecha": se guardan como texto `AAAA-MM-DD`, que por suerte se ordena bien. Para sacar el año o el mes se usa `strftime`: `strftime('%Y', fecha)` da el año, `strftime('%Y-%m', fecha)` el año-mes. Devuelve *texto*; si lo necesitás como número, `CAST(... AS INTEGER)`.

CASE WHEN: decidir fila por fila

`CASE` es el "si... entonces... sino" de SQL. Sirve para clasificar, y también para **contar o sumar solo lo que cumple una condición** dentro de un `GROUP BY`:

```
SELECT mes,
       COUNT(*) AS cuotas,
       SUM(CASE WHEN fecha_pago IS NULL THEN 1 ELSE 0 END) AS impagas,
       ROUND(100.0 * SUM(CASE WHEN fecha_pago IS NOT NULL THEN 1 ELSE 0 END) /
COUNT(*), 1) AS pct_pagadas
FROM cuotas
GROUP BY mes;
```

mes	cuotas	impagas	pct_pagadas
2025-03	8	1	87.5
2025-04	8	3	62.5

Este patrón (un `CASE` adentro de un `SUM` o un `AVG`) es la forma típica de calcular **porcentajes**, como "qué porcentaje de las cuotas se pagó".

Pasos siguientes: subconsultas, CTE y funciones de ventana

Cuando una consulta necesita un resultado intermedio, podés ponerle nombre con `WITH` (una **CTE**) y usarlo después como si fuera una tabla. Y cuando necesitás "numerar" o comparar filas *dentro de un grupo* sin colapsarlas, existen las **funciones de ventana**, con la forma `RANK()` `OVER (PARTITION BY grupo ORDER BY criterio)`. No hace falta dominarlas antes del taller: en la **TEORIA.md** hay ejemplos resueltos para cuando las necesites.

Pensar como analista, no solo escribir consultas

Escribir SQL es la parte fácil. Lo difícil es que el número **sea el correcto**. Los datos reales vienen sucios: hay huecos, valores escritos de distintas formas, filas repetidas por un `JOIN`. Por eso el proyecto del taller pide hacer cuatro cosas, siempre:

1. **Traducir la pregunta de negocio a una definición.** "Puntualidad" o "ocupación" no significan nada hasta que alguien decide cómo se calculan: ¿qué cuenta como puntual?, ¿qué pasa con los vuelos cancelados? No hay una respuesta única, pero sí una regla: **escribirla y aplicarla igual en todas las consultas** (en el taller se llama *diccionario de métricas*).
2. **Explorar antes de preguntar.** Mirá las columnas que vas a usar: qué valores distintos tienen, cuáles son el mínimo y el máximo, si hay `NULL`. Dos minutos de exploración te ahorran una hora de números raros.
3. **Verificar con controles cruzados.** ¿La suma de las partes da el total? ¿Algún porcentaje pasa de 100? ¿El total cambió después de un `JOIN`? Si algo no cierra, no lo entregues: buscá por qué.
4. **Contar qué encontraste.** Un número suelto no sirve; sirve "el 40 % de las demoras se concentra en X, y recomendando Y".

Trabajar con IA

En el taller vas a poder usar una IA para escribir consultas. Puede ahorrarte mucho tiempo, pero con un límite importante: **la IA no conoce tus datos**. Si le pedís algo sin darle contexto, va a inventar nombres de columnas o a usar la sintaxis de otro motor. Reglas simples:

- **Pasale siempre el esquema** (el botón "Copiar esquema para la IA" del Lab lo hace por vos) y aclarale que el motor es **SQLite**.
- **Explorá los datos primero** y contale lo que encontraste (por ejemplo, qué valores tiene una columna).
- **Leé la consulta antes de ejecutarla**, en el orden en que SQLite la resuelve: `FROM` → `JOIN` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `ORDER BY`. Pedile que te la explique línea por línea.

- **Verificá el resultado**, siempre. Una consulta que corre sin error no es una consulta correcta.

Hay una guía completa de esto (**GUIA_IA.md**) en el proyecto.

Y cuando los datos están mal armados: normalización

A veces una tabla está pensada de una forma que genera problemas: el mismo dato repetido en muchas filas, columnas como `causa_1` , `causa_2` , `causa_3` , o el mismo concepto escrito de tres maneras distintas. **Normalizar** es reorganizar esa tabla para que *cada dato viva en un solo lugar*, repartiéndolo en varias tablas unidas por claves. Las "formas normales" son reglas para detectar qué está mal:

Forma	Regla en una frase	Se rompe cuando...
1FN	Una celda, un valor.	Hay listas dentro de una celda o columnas repetidas (<code>causa_1</code> , <code>causa_2</code>).
2FN	Cada dato depende de la clave <i>entera</i> .	La clave tiene varias columnas y un dato depende solo de una de ellas.
3FN	Los datos dependen de la clave, no de otro dato.	Hay cadenas: la matrícula determina el modelo, y el modelo determina la capacidad.

Si llegás a la parte de normalización, vas a limpiar una tabla desordenada, diseñar las tablas nuevas, mover los datos con `INSERT INTO ... SELECT DISTINCT ...` y comprobar con un `JOIN` que no perdiste nada. En la **TEORIA.md** hay un ejemplo resuelto de punta a punta con el club de barrio.

Cheatsheet

Quiero...	Uso...
Ver unas filas de una tabla	<code>SELECT * FROM tabla LIMIT 10;</code>
Contar filas	<code>COUNT(*)</code>
Un resumen por grupo	<code>GROUP BY</code>
Filtrar filas / filtrar grupos	<code>WHERE</code> / <code>HAVING</code>
Juntar dos tablas	<code>JOIN ... ON a.id = b.a_id</code> (o <code>LEFT JOIN</code>)
Saber si falta un dato	<code>IS NULL</code> / <code>IS NOT NULL</code>
Calcular un porcentaje	<code>100.0 * a / b</code> (¡con decimal!)
Contar solo lo que cumple algo	<code>SUM(CASE WHEN ... THEN 1 ELSE 0 END)</code>
Sacar el mes de una fecha	<code>strftime('%Y-%m', fecha)</code>
Pasos intermedios con nombre	<code>WITH nombre AS (...)</code>
Numerar filas dentro de un grupo	<code>RANK() OVER (PARTITION BY ... ORDER BY ...)</code>
Deshacer un desastre	Botón Reiniciar base del Lab

Con esto alcanza para arrancar. En el taller van a equivocarse, y está perfecto: **un error de SQL no rompe nada**, y los mensajes del Lab te explican qué pasó. Lo importante es leer lo que devuelve la consulta y preguntarse si tiene sentido.